

TECHNICAL PORTFOLIO · 2026 · PERSONAL REPOSITORIES

보안 × AI 엔지니어 김강민

SIEM · SOAR · EDR · 포렌식을 제품에서 개발하고, 같은 것을 개인 서버에 처음부터 조립했다

이 데크의 모든 자료는 개인 저장소(github.com/adorahelen · 26개)에서 나온다 — 회사 산출물 0건

SIEM / SOAR

EDR · 엔드포인트

디지털 포렌식

온프레미스 LLM

Go · Python · C++

홈서버 운영

프로필 — 실무에서 만드는 것을, 집에서 처음부터 조립한다

SIEM·SOAR 솔루션 기업의 보안/AI 엔지니어. 경력 1년 3개월.

"도구를 써봤다"가 아니라 "같은 것을 직접 만들어 한계를 확인했다"가 이 텍스트의 내용.

현 재직사 — SIEM/SOAR 솔루션 기업 (2026.02 ~)

AI 보안 엔지니어 · 7인 팀

관측성 플랫폼과 LLM 어시스턴트 개발. 담당 서술은 이력서 참조 —
제품 코드·구조·화면은 이 텍스트에 일절 넣지 않는다.

전 직장 — DLP·데이터 보안 기업 (2024.12 ~ 2025.08)

보안 엔지니어

개인정보 탐지 솔루션. 대용량 파일 API Redis 캐싱으로 응답 80% 단축,
NER 기반 비정형 데이터 탐지 범위 확장

학술 · 수상

제1저자 논문 — "디지털 포렌식 관점에서의 Threads 사용자 행위 분석"
한국정보보호학회 동계학술대회(CISC-W'25) 구두발표 · 논문번호 328

KISTI 한국과학기술정보연구원 원장상 수상

어학 — JLPT N1

개인 저장소 실측 (adorahelen 26개)

본인 코드 286,062줄 — 벤더 라이브러리·업무 산출물 제외 후 전수 클론
계측

코드 파일 1,801 · 테스트 165 · 문서(MD) 973 · GitHub Actions 5개
저장소

프로젝트 맵 — 개인 저장소 6선 (전부 adora helen)

SIEM-Trinity 자작 XDR

홈서버 1대에 수집·탐지·분석·대응 4계층 + SOC 콘솔을 올린 모노레포. 공개판 siem-trinity-public 별도 유지

자동대응 6단계 · 컨테이너 10

abliteration 재현 AI 보안

LLM 거부 정렬이 단일 방향으로 무너지는지 논문 재현·측정. 거부 방향을 탐지 신호로 역이용하는 방어 관점

거부율 1.00→0.00 · 공개 발행

edr-lab 엔드포인트 센서

커널 드라이버 없이 Ring 3에서 어디까지 관측되는가. 프로세스·파일·네트워크·레지스트리 + 포렌식 수집

C++ 2,840줄 · 규칙 14종

security-labs 포렌식 · 암호

Threads 앱 포렌식(수상 논문 기반) · Windows 브라우저 포렌식 · AES-128 직접 구현

KISTI 원장상 · CAVP 검증

agent-console 온프레미스 AI

엔진은 고정, 도메인은 카트리지(프롬프트·지식·모델)로 교체하는 에이전트 콘솔. 보안관제 도메인 카트리지 포함

Python 70,424줄 · 테스트 55

운영 · 인프라 fossil-fuel · ops

AWS 5개월 운영 → 미니 PC 1대 이전(고정비 0원). 홈서버 인시던트 6건·런북 9종 기록

쓰기증폭 700배 사고 해결

SIEM-Trinity — 홈서버 1대로 굴리는 자작 XDR

수집·탐지·분석·대응 4계층과 SOC 콘솔을 한 모노레포에 넣고, 상용 XDR이 하는 일을 오픈소스로 재현

- 4계층 모노레포 — 01 수집(Zeek·Suricata·Wazuh) / 02 탐지(FastAPI BFF·ML 4종) / 03 분석(Ollama·ChromaDB) / 04 UI(TrinitySOC)
- XDR 자동 대응 체인 6단계를 코드로 완성 — Wazuh 가시성 → fail2ban 차단 → active-response → MISP IOC → Shuffle SOAR → TheHive 케이스
- 6단계 전부 구현했으나 운영 토글은 안전상 OFF(dry-run) — 오탐 자동 차단이 자기 서비스를 끊는 위험을 인정하고 기본값을 보수적으로 잡았다
- LLM 알람 분석은 4섹션 구조 프롬프트(요약·공격체인·위험평가·권장대응)로 고정해 환각을 구조로 억제
- 공개판 siem-trinity-public 별도 유지 — 1커밋 스쿼시 + 운영 호스트 자산 정보 제거 후 분리

핵심 성과 · 실측

- 컨테이너 10개 상시 가동, 외부 노출 포트는 콘솔 1개뿐 — 나머지 전부 localhost 바인딩
- ATT&CK 기법 697건을 ChromaDB에 적재해 탐지 결과와 매핑
- TrinitySOC — React 18 + TypeScript strict, 위젯 23종 전부 CRUD·드래그 배치
- detection-api FastAPI 엔드포인트 28개가 Loki·Prometheus를 단일 BFF로 통합
- 탐지 로직은 Isolation Forest·변동계수·엔트로피 등 4종을 병렬로 두고 교차 확인

STACK

Python 3.12

FastAPI

Loki

Prometheus

Wazuh

Suricata

Zeek

scikit-learn

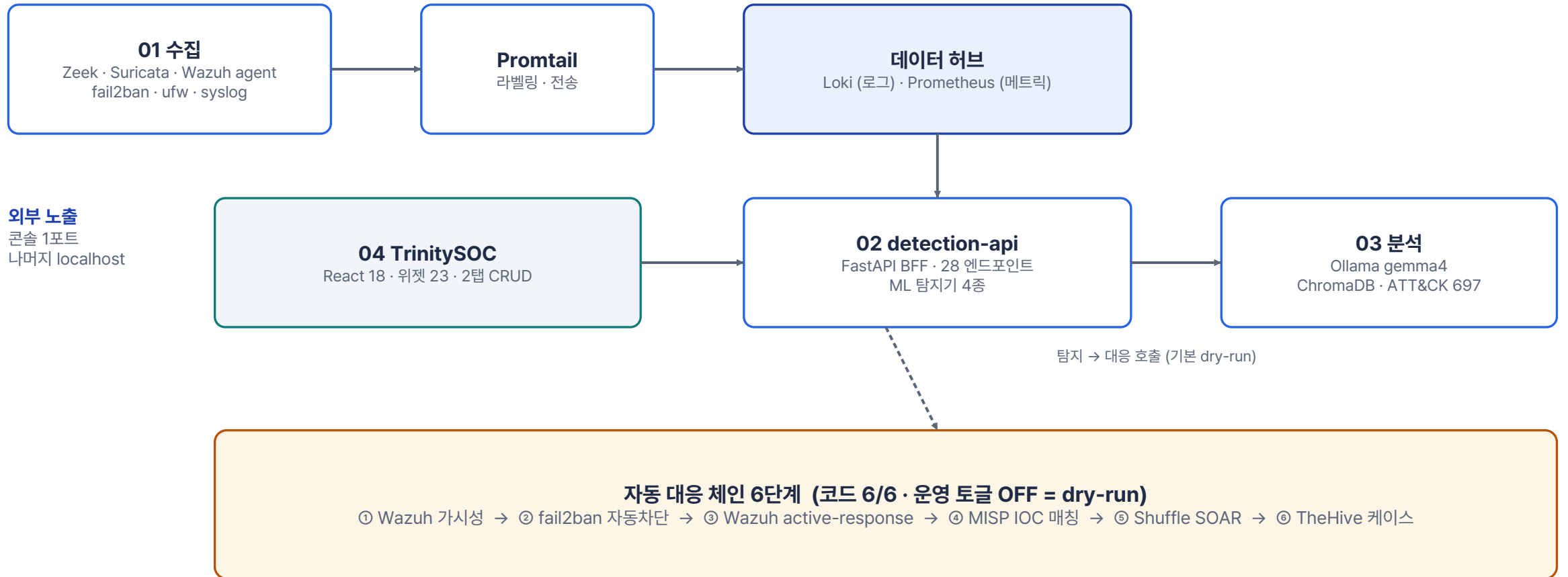
Ollama

ChromaDB

React 18

Docker Compose

SIEM-Trinity — 아키텍처



LLM 정렬 취약성 실증 — ablation 재현

"거부는 잔차 스트림의 단일 방향이 매개한다"(Arditi et al., 2024)를 직접 재현해, 모델의 안전이 얼마나 얇은 계층인지 수치로 측정

- 거부 방향 추출 — 매칭 프롬프트 쌍(유해 24 / 무해 24)의 마지막 토큰 잔차 평균차를 레이어별로 정규화
- 레이어 스윙 0.3·L ~ 0.8·L — 후보 6곳에 추론-time ablation을 걸고 분리한 검증셋(12개) 거부율로 best 선택
- 3단계 측정 — BEFORE → 추론 ablation → 영구 weight 직교제거(o_proj·down_proj 72개 행렬)
- 책임 설계 — 유해셋은 거부율만 집계하고 생성 텍스트는 저장하지 않으며, 수술된 가중치는 배포하지 않는다
- 영문·국문 글로 발행 — 이 텍스트의 프로젝트 중 유일하게 외부에서 검증 가능하다

핵심 성과 · 실측

- 거부율 1.00 → 0.00 (추론 ablation) / 0.04 (영구 수술) — 재학습 없이 방향 하나 제거로
- 후보 레이어 6곳 중 3곳은 효과가 정확히 0 — 스윙을 안 하면 "안 뚫린다"는 위음성이 나온다
- 거부율 0% ≠ 지식 해금 — 이 모델은 수술 전에도 정당한 보안 질문에 전부 답했다(과잉거부 0.00)
- 조잡한 제거는 모델을 손상시킨다 — 17×23을 "39"로 답하고, 손상이 하필 공격·보안 주제에 집중
- 거부율만 보면 안 된다 — 전후 모두 0.00인데 실제 답변은 XSS·EternalBlue·CSRF에서 무너져 있었다

STACK

Qwen2.5-3B-Instruct

PyTorch

transformers

잔차 스트림 개입

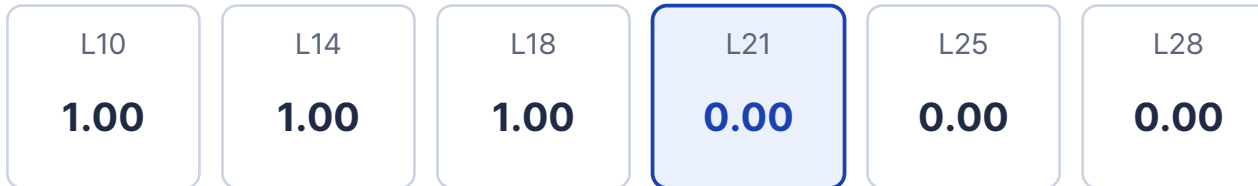
forward hook

직교 투영

골든셋 평가

abliteration — 무엇을 측정했나

레이어 스윕 — 거부가 매개되는 지점 찾기



앞·중간 레이어(10·14·18)에서는 효과가 정확히 0 — 튜닝을 안 해서 생긴 위음성이다
 거부 방향은 네트워크 후반부(≈ 0.6 depth)에서 매개된다 → **best = L21**

거부율 — 3단계 측정

단계	유해	과잉거부
BEFORE (원본)	1.00	0.00
추론 ablation (L21)	0.00	0.00
영구 weight 수술	0.04	0.00

과잉거부는 처음부터 끝까지 0.00 — 벗겨진 것은 행동 족쇄이지
 능력이 더해진 게 아니다

방어 산출물로의 전환 — 같은 벡터가 공격에도 방어에도 쓰인다

제거 대상이던 거부 방향 r 을 추론 시 활성 모니터로 쓰면, 잔차 스트림을 r 에 투영한 값이 탈옥·우회 시도를 알아채는 실시간 탐지 신호가 된다.
 결론은 "정렬은 뚫린다"가 아니라 — rank-1 편집으로 지워지는 것을 보안 경계로 삼으면 안 되고, 통제는 지워질 수 없는 곳(API·검색 계층·출력 필터)에 두어야 한다.

edr-lab — 커널 없이 어디까지 관측되는가

상용 EDR이 커널 드라이버로 하는 일을 User-mode(Ring 3)에서 재현하고, 안 되는 것은 안 되는 이유까지 남긴다

- 센서를 C++ 2,840줄로 구현 — 프로세스(WMI Trace)·파일(ReadDirectoryChangesW)·네트워크(GetExtendedTcpTable)·레지스트리(RegNotifyChangeKeyValue) 관측
- 커널 콜백마다 Ring 3 우회로를 대응시키고 그 대가를 표로 명시 — 파일 변경은 유발 PID를 알 수 없고, 네트워크는 3초 폴링 사이 단명 연결을 놓친다
- 규칙 엔진(*.rule) 14종 + 부모-자식 프로세스 상관까지 동작 → DETECTION 알림. 다중 이벤트 시간 윈도우 상관은 미구현으로 명시
- 포렌식 수집 모드 별도 — \$MFT · 레지스트리 하이버 · .evtx · Prefetch를 매니페스트와 해시로 수집
- Sysmon 14개 이벤트 기준으로 커버리지를 자체 채점 — 6종 미커버(DLL 로드·인젝션·lsass 접근·DNS·드라이버·WMI 구독)를 사각으로 공개

핵심 성과 · 실측

- 이벤트 → SQLite(WAL) 적재 → Electron 콘솔이 1초 증분 폴링, Windows 실행기 중단 확인
- 런타임 의존성 없는 정적 링크 · MSVC / MinGW-w64 양쪽 툴체인 빌드 검증
- 저장소 C/C++ 총량은 298,925줄이지만 본인 코드는 2,840줄 — 나머지는 벤더(SQLite·httplib) 정적 동봉. 면접에서 절대 부풀리지 않는다
- 두 저장소(센서·콘솔)를 커밋 히스토리 보존한 채 통합 — "센서는 있는데 볼 방법이 없던 상태"를 잇는 것이 목적
- 이름이 실물을 앞서지 않도록 README에 "아직 EDR이 아니다"를 E·D·R 글자별로 자기 채점

STACK

C++17

Win32 API

WMI

SQLite (WAL)

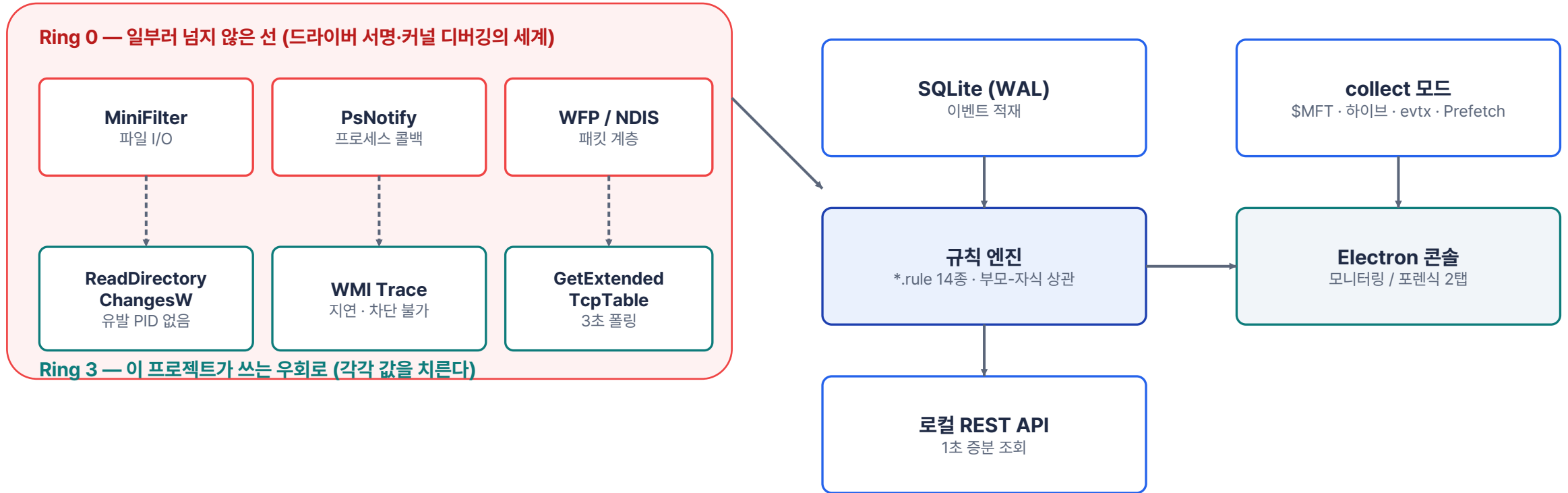
Electron 39

TypeScript 5.9

CMake

MSVC · MinGW-w64

edr-lab — 아키텍처



Sysmon 14개 이벤트 기준 자체 채점 — 5 커버 · 3 부분 · 6 미커버

남은 사각의 핵심: ID 7 모듈 로드 · ID 8 인젝션 · ID 10 lsass 접근

security-labs — 디지털 포렌식 · 암호 구현

학회 발표 논문의 기반이 된 앱 포렌식 실습과, 표준 테스트 벡터로 검증한 암호 구현을 한 저장소로 합쳤다

- Threads 앱 포렌식 — 루팅 Android 13 기기에서 adb+tar로 추출, 4단계 파이프라인(행위 모방 → 추출 → 디렉터리 분석 → 아티팩트 수집)
- 핵심 발견 — mdcore_messages의 data BLOB에 DM 본문이 평문 저장. 방·시각·발신자·본문이 모두 복원된다
- WAL(dmm_database-wal)을 빼면 최신 메시지를 놓친다 — 본 DB만 보는 절차의 공백을 지적
- 선행연구 3편이 "모바일 분석은 수확이 적다"고 평가한 지점을, 최신 버전에서 평문 복원까지 해내 뒤집었다
- 같은 저장소에 Windows 브라우저 포렌식(VMware 이미징 + Sleuthkit inode 추적)과 AES-128 C 직접 구현을 함께 둔다

핵심 성과 · 실측

- 한국정보보호학회 동계학술대회(CISC-W'25) 제1저자 · 논문번호 328 · 구두발표 세션 「디지털 포렌식 I」
- KISTI 한국과학기술정보연구원 원장상 수상
- 복원되는 사용자 행위 — 열람 게시글과 시점, DM 본문·발신자·시각, 미전송 대기 이미지, 인앱 검색어와 검색 시각
- AES-128 ECB를 라운드 함수부터 구현 후 OpenSSL 대조, NIST CAVP KAT 벡터 자동 검증기까지 작성
- 한계를 명시 — Android 한정 · 정적 추출 중심. iOS 확장과 동적/네트워크 포렌식을 과제로 남김

STACK

C

adb

SQLite

Sleuthkit

EnCase

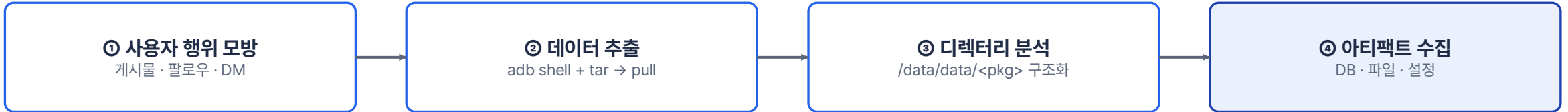
DB Browser

VMware

OpenSSL

NIST CAVP

Threads 앱 포렌식 — 아키텍처



databases/
mdcore_messages
DM 본문 평문 · thread_id · timestamp · 발신자

databases/
contact_database
팔로잉 관계 · 내부 ID ↔ 계정 매핑

databases/
barcelona_feed_items
열람한 게시물과 열람 시점

databases/
dmm_database-wal
본 DB 미반영 최신 메시지 · 전송 이미지 경로

files/ · cache/
creation_file_manager · tmp_photo
전송 이미지 · 미전송 대기 이미지 (파일명에 시각)

shared_prefs/
USER_PREFERENCES.xml
인앱 검색어 전체 + 검색 시각 · 종료 직전 화면

노란 칸이 이 연구의 핵심 — 상용 도구가 보지 않는 WAL과, 암호화됐을 것으로 여겨지던 DM 본문

agent-console — 카트리지형 온프레미스 AI 에이전트 콘솔

엔진(RAG·intent·멀티백엔드 서빙)은 고정하고, 도메인·지식·모델 세 슬롯만 카트리지로 갈아 끼운다

- 도메인에 종속된 에이전트 구조에서 도메인만 건어내 범용 엔진으로 재설계 — 건어낸 도메인은 첫 카트리지로 보존해 "도메인 전체가 카트리지 하나"임을 실증
- 3슬롯 설계 — 프롬프트(config 경로 18개 외부화) · 지식(bulk upload → 임베딩 → Qdrant) · 모델(config 한 줄). 슬롯 교체에 코드 수정이 없다
- install.sh가 VRAM·RAM을 감지해 HW 티어를 판정하고 프리셋을 자동 선택 — 16GB 장비는 26B + n-cpu-moe 오프로딩으로 확정
- 카트리지 CLI 완비(validate·mount·unmount·list·status·purge) · 검증 완료 3종 — 보안관제 · 개발도우미 · 개인정보/ISMS-P
- 지식의 출처를 파일 단위로 역추적 — 보안관제 3,955건 = 공개 코퍼스 1,575 + 자체 1,930 + LLM 301. 라이선스 실사까지 문서화

핵심 성과 · 실측

- Python 70,424줄 · 테스트 파일 55개 — 두 계정 전체 저장소 중 최대·최다
- 보안 미결 항목을 스스로 공개 — PII 마스킹이 특정 핸들러만 경유해 API 티어에선 원문이 나간다는 점을 기록
- RAG는 BGE-M3 dense + ColBERT 2-way RRF 리랭킹. 전략을 바꿔가며 비교한 실험 흔적이 파일로 남아 있다
- 멀티백엔드 서빙 — 로컬 llama-server와 외부 API를 같은 인터페이스로, OpenAI 호환 엔드포인트 제공
- install 원클릭 → systemd 등록까지 종단 자동화, T1·T2 티어 실측 검증 완료

STACK

Python 3.11+

FastAPI

llama.cpp

Qdrant

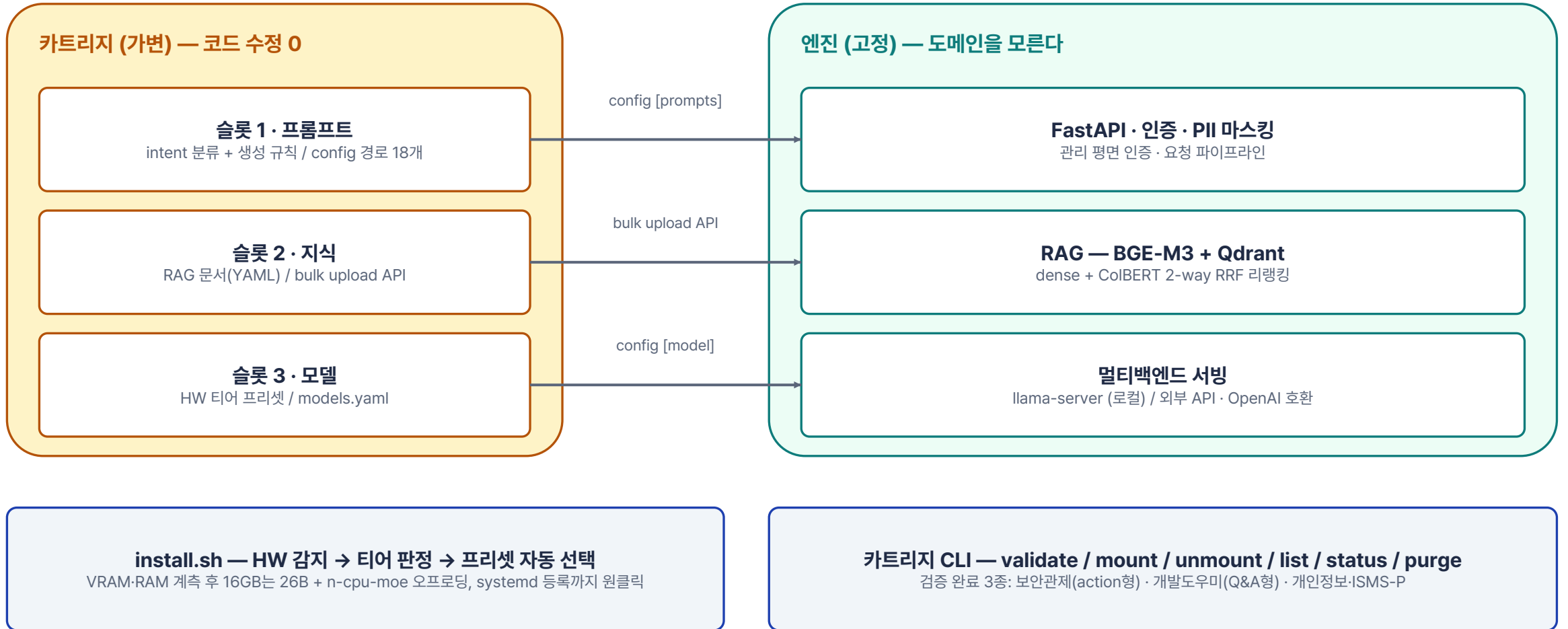
BGE-M3

RRF 리랭킹

systemd

OpenAI 호환 API

agent-console — 아키텍처



운영 · 인프라 — 클라우드에서 내려와 직접 굴린다

매니지드가 대신 해주던 일의 목록을 만들고, 홈서버 한 대에서 장애를 겪고 근본 원인까지 파고든 기록

- fossil-fuel — AWS(EC2·RDS·ALB)로 5개월 운영하던 학과 동아리 서비스를 프리티어 종료를 계기로 미니 PC 1대로 이전. 고정 운영비 월 0원
- 1세대(AWS 179커밋) 히스토리를 legacy 브랜치로 보존하고, 이전 중 드러난 결함 수정을 2세대(78커밋)의 주 내용으로 삼았다
- kangminlog-ops — 홈서버 1대의 절차·인시던트·자동화를 단일 저장소로 추적. 인시던트 6건 · 런북 9종
- 대표 사고: ClickHouse 쓰기 증폭 700배 — 6.9일간 NVMe에 327GB 쓰기. 실제 사용자 데이터는 0.8%(26MiB)뿐이고 99.2%가 ClickHouse 자기 로그였다 (원인 분해는 다음 장)
- dodgers-labs — 공격면이 큰 컴포넌트(파일 파서·LLM 추론·인증 DB)만 분리해 상시 노출하지 않고 온디맨드로만 기동하는 구조

핵심 성과 · 실측

- NVMe 쓰기 -97% — 배치 확대 · system 로그 정리 · TTL 설정 후 실측
- 다른 서버의 NVMe 장애 사례와 홈서버 사고를 대조 분석해 같은 패턴임을 문서화 (식별 정보 제거 후)
- 백업 cron이 조용히 실패하던 것, 자동 PR이 무음으로 실패하던 것 — "알림 없는 실패"를 인시던트로 별도 기록
- CI는 H2로 DB 없이 기동하도록 구성해 외부 의존 없이 검증
- dodgers-labs 전 서비스에 OpenTelemetry 트레이싱 적용, PII 탐지·마스킹은 한국어 NER + 정규식 병행

STACK

Java 17

Spring Boot 3.5

PostgreSQL 16

Docker Compose

ClickHouse

OpenTelemetry

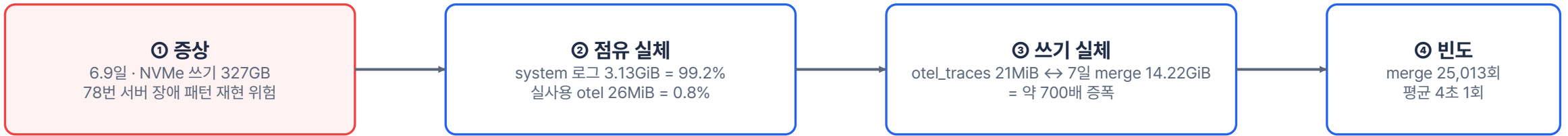
restic

systemd

Tailscale

iptables · UFW

ClickHouse 쓰기 증폭 700배 — 진단에서 조치까지 — 아키텍처



이 사고를 회사 서버 NVMe 장애 사례와 대조해 같은 패턴임을 문서화했다 — 회사 식별 정보는 제거한 뒤에만 기록한다.

침해사고 대응 실사례 — 내가 만든 시스템의 결함을 내가 찾았다

시험용으로 띄워 둔 수신 바이너리가 모든 인터페이스에서 인증 없이 로그를 받고 있었다. 탐지부터 재발 방지까지 전 과정을 문서로 남겼다.



증거 4건으로 입증

- ① 임의 JSON을 수신 엔드포인트로 전송 → 정상 수락 + 파일 저장 재현
- ② 바이너리 문자열 검사 — auth · token · api_key · bearer · secret 패턴 0건. 인증 로직 부재를 코드가 아닌 산출물로 입증
- ③ 외부에서 수집된 Windows 이벤트 로그 17,276건이 평문 저장돼 있음을 확인
- ④ 로그인/로그오프 이벤트와 사용자 SID·프로세스명 등 민감 필드가 원문 그대로 포함

왜 이 사례를 넣는가

탐지 → 증거 → 영향 산정 → 조치 → 재발 방지의 전 과정이 남아 있다.
 침해사고 대응(IR) 절차 그 자체다.
 남의 사고가 아니라 내가 만든 시스템의 결함을, 내가 찾아 기록했다. 자기 결과물의 취약점을 공개 기록으로 남기는 쪽을 택했다.
 사고 문서에는 공인 IP와 수집된 이벤트 필드가 들어 있어, 저장소 공개 시 마스킹이 선결 조건임을 함께 기록해 두었다.

SUMMARY

스택 총괄 · 일하는 방식

보안

SIEM (Loki·OpenSearch)

SOAR (Shuffle·플레이북)

EDR (User-mode 센서)

HIDS (Wazuh)

NIDS (Suricata·Zeek)

TI (MISP·TAXII/STIX)

케이스관리 (TheHive)

MITRE ATT&CK

디지털 포렌식

AES 구현·CAVP 검증

fail2ban·UFW·iptables

AI / LLM

llama.cpp

Ollama

vLLM

Qdrant

ChromaDB

BGE-M3

RRF 리랭킹

QLoRA / PEFT

온디바이스 LLM

PII 마스킹

프롬프트 보안

언어 · 백엔드

Python 142,550줄

TypeScript 67,134줄

Go

Shell 19,711줄

Java 11,456줄

C++ 2,840줄 (본인)

FastAPI

Spring Boot

React

데이터 · 인프라

OpenTelemetry

ClickHouse

Prometheus

PostgreSQL

Redis

Docker Compose

systemd

nginx

restic

Tailscale

GitHub Actions

정확함 > 영리함

스펙을 읽고 경계를 검증하고 운영에서 확인한다

과장하지 않는다

edr-lab 30만 줄 중 본인 코드 2,840줄임을 먼저 적는다

안 되는 것도 결과물

Ring 3의 한계·미커버 이벤트를 사각으로 공개한다

재현 가능성

원클릭 설치·회귀 테스트·5축 문서를 저장소마다 유지한다

보안 스택을 설계부터 운영까지 직접 만들어 본 엔지니어입니다.

SIEM·SOAR·EDR·포렌식을 제품에서 개발하고, 같은 것을 개인 서버에 오픈소스로 조립해 무엇이 되고 무엇이 안 되는지를 코드로 확인합니다.

GitHub github.com/adorahelen

공개 저장소 github.com/adorahelen/siem-trinity-public · [/ai-console-public](https://github.com/adorahelen/ai-console-public)

Email adorahelenmin@naver.com

위 2건은 로그인 없이 바로 열람 가능합니다 — 나머지 저장소는 private이라 요청 시 초대해 드립니다.